BebopBot: Jazz Lick Generator with LSTM

Euy Shick Hong
Institute for Computer Music and Technology (ICMT)
Seoul, Republic of Korea
antaresax@gmail.com

Abstract

Improvisation is one of the important features of jazz. Most jazz musicians study harmony theory and analyze jazz players' recordings to enhance their approach to improvisation. However, it takes a long time and considerable effort to learn and memorize jazz licks for improvisation. In this paper, we explore the use of jazz lick generators for jazz improvisation. This system generates jazz licks, leveraging an LSTM model and web technologies. Generated jazz licks can be utilized through various approaches. We present how players can interact with the system and use the licks in jazz improvisation.

Keywords: Jazz Improvisation, Jazz Lick Generation, LSTM, Web Audio API, Human-AI Interaction, Interactive Music Systems

1 Introduction

Improvisation is one of the main features of jazz, especially in bebop and hard bop (extended improvisation, fast tempos). Therefore, many jazz players learn and study jazz licks from recordings, creating new licks.

However, considerable time and effort are required as follows in order to utilize jazz licks in live jazz performance through the study of jazz licks. First, jazz musicians extract licks by analyzing the chords in jazz recordings. Second, the extracted licks are recreated and memorized. Third, the memorized licks are practiced with varying tempos and rhythms. Finally, jazz musicians apply the practiced licks in live jazz performance. Accordingly, this paper aims to improve the efficiency of utilizing the jazz licks through a web-based jazz lick generator, leveraging AI technology (Long Short-Term Memory, LSTM).

The web-based jazz lick generator enables users (e.g., jazz musicians, composers, others) to generate licks through mobile and PC; the licks are generated and saved with one click. The generated licks can also be played using the Web Audio API and utilized through various approaches for motif ideas and recreation.

This paper explores how users can interact with the system through the following pipelines:

1. Generate/save → Memorize → Play
2. Generate/save → Recreate → Memorize → Play
3. Generate/save → Use the motif → Recreate → Memorize → Play

This approach focuses on interaction and extension between users and the system. The proposed system provides the following contributions.

1. Web-based access on mobile or desktop.
2. Player interaction with the system, including saving, memorizing, and developing motif ideas.

We first discuss relevant studies in the following section.

2 Related Work

2.1 Web-based application

Recently, web-based technologies (HTML5, Web Audio API and fast browser engines) have been rapidly developing, providing various types of web-apps, especially, enabling audio and video interaction in real-time.

WAAX is a JavaScript library (based on the Web Audio API), facilitating music work (with creation and performance), audio programming (for rapid development) with useful features [1]. In 2017, Webxophone presented a web-based wind instrument application (without installation) like saxophone through mobile technologies, harnessing microphone input (for blowing), multitouch (for fingering 6 buttons), gyro sensors, sound processing and synthesis in real-time, Web Audio API for musical expression and various musical performances [2]. In parallel, in 2017, Trackswitch.js is a versatile web-based audio player (manageable software tool for non-expert developers and inexperienced users), allowing researchers to present results (from audio processing), and listeners to switch between multiple audio tracks [3]. Furthermore, there have been studies on the Web Audio API, as well as introductions to interactive platforms [4]. In addition, Turchet et al. presented a web-based musical system designed for vocal ensembles, utilizing it to augment the practice and experience of choir singing, leading to novel forms of musical expression [5]. Moreover, Ma et al. introduced a textile-based human-AI system used both for musical performance and public interaction, examining the potential of soft materiality for dynamic human-AI interactions [6]. In 2025, Smith explored the use of GestAlt, an AI-based interactive music system designed to adapt to users, collaborating with a performer by gestures and motion to generate audio changes [7]. More recently, Aynsley presented various strategies using AI text-to-image (TTI) generators in the conceptual design of new musical instruments through workshops [8].

2.2 AI-based works

RNN (Recurrent Neural Network) models have been widely used in sequence generation tasks such as music and text generation. In particular, LSTM models have been studied for capturing long-term dependencies, including music and text generation, harmony progression, among others (more effectively than RNNs). Furthermore, sequence generation tasks have recently expanded to Transformer models with high performance.

In 2023, Wen presented a new time series forecasting model LSTM-attention-LSTM that used two LSTM models as the encoder and decoder (attention mechanism) and assessed the effect of the attention mechanism at different time steps by varying the time step [9]. Subsequently, Al-Selwi presented a systematic literature review (SLR) using an in-depth four-step approach based on the PRISMA methodology, addressing the weight initialization and optimization techniques for RNN-LSTM performance, providing a roadmap to enhance RNN-LSTM networks [10]. In 2025, Alizadegan compared LSTM and bidirectional LSTM for time series analysis to improve forecasting accuracy and the study showed superior performance compared to several traditional forecasting methods [11]. At the same time, Azimi et al. presented a pipeline to integrate a fine-tuned open-source text-to-audio model into Ableton Live for the improvisation of contemporary electronic music, generating audio fragments based on text prompts, enabling dynamic interaction [12].

So far, we have reviewed various AI-related works. This paper aims to design and develop a web-based application (for sequence generation) using a time series-based LSTM. Although recent AI models such as Transformers are available, there are several reasons for choosing LSTM, as follows.

First, LSTM is relatively lightweight compared to Transformer models, resulting in faster loading times and easier deployment and operation in web-based environments.

Second, using real data, we design the model, make predictions, and implement the entire process to generate the results, evaluating the merits and limitations.

Third, through this study, we aim to validate the potential of jazz lick generation and evaluate its effectiveness.

Therefore, we will sequentially investigate RAG and Transformer-based approaches in future work after the LSTM-based approach.

3 Design and Implementation

In this section, we examine the design, architecture and implementation of an interactive web app BebopBot for jazz improvisation using web and AI technologies.

3.1 System Design Architecture

BebopBot is a web-based application that generates jazz licks (displayed as musical notation and saved to storage) using an LSTM (harnessing WJAZZD Dataset) on a Python-based server. The generated jazz licks are played back through the Web Audio API (DrSax.js [13]), allowing various sound synthesis.



Fig. 1 BebopBot on Mobile Device

3.2 Web Interface

BebopBot provides a UI/UX environment (lick generation, musical notation display, sound playback and sound synthesis as shown in Figure 1) for jazz lick learning and creation; users can easily access BebopBot through web browsers on mobile and desktop and repeatedly generate new jazz licks using the Generate button.

The generated new jazz licks are displayed as musical notation and can be played using the Playback button. BebopBot also allows users to save the generated jazz licks for practicing and creating.

The BebopBot system consists of three components as follows.

Table 1 Main Components of BebopBot

Component	**Function**
Lick Generation	Jazz lick generation and storage
Display and Playback	Musical notation visualization and audio playback
Synthesis Control	Oscillator type selection and timbre parameter control



Fig. 2 The components of BebopBot.

3.3 AI Generation Model

- **Jazz Dataset**
  This study uses the data from Weimar Jazz Database (WJAZZD): pitch, duration, bar, beat and chord [14]. We examined and limited the data to the dominant chord that allows various scale approaches (whole-tone, diminished, altered, harmonic 5 down etc.). A total of 2271 samples were processed and normalized to C dominant 7 chord.
- **LSTM Model**
  In this study, we designed the LSTM architecture. The model receives a sequence of three input notes and predicts the next note. The model structure is as follows.

$$\begin{gathered}\text{Input (3 notes)} \\ \downarrow \\ \text{LSTM (128 units)} \\ \downarrow \\ \text{Dropout (0.2)} \\ \downarrow \\ \text{Dense Layer (26 classes)} \\ \downarrow \\ \text{Softmax}\end{gathered} \tag{1}$$

- **Model Training**
  Total 2,271 samples were used for model training (with Adam optimizer, Categorical Crossentropy loss, a learning rate of 0.001 and 100 epochs). Early Stopping and ReduceLROnPlateau were applied (during the learning process). Training was stopped at Epoch 25. The epoch 10 (lowest Validation Loss) model was used.

3.4 Lick Generation Process

The jazz lick generation process is as follows.

$$\text{Input} \rightarrow \text{LSTM Prediction} \rightarrow \text{Next Note Selection}$$
$$\rightarrow \text{Sequential Generation} \rightarrow \text{8-Note Lick}$$

LSTM model predicts next note after receiving three input notes. Generated note is used as the next input. This process is repeated five times to generate a total of eight notes as follows.

Step 1: $[11, 20, 6] \rightarrow 20$
Step 2: $[20, 6, 20] \rightarrow 21$
Step 3: $[6, 20, 21] \rightarrow 2$
Step 4: $[20, 21, 2] \rightarrow 16$
Step 5: $[21, 2, 16] \rightarrow 15$

The final generated lick is:

$$[11, 20, 6, 20, 21, 2, 16, 15]$$

3.5 System Implementation

The overall programming architecture of BebopBot is shown in Figure 3. BebopBot was developed using Python and Django framework for the server and API while Keras and TensorFlow were used for model training.

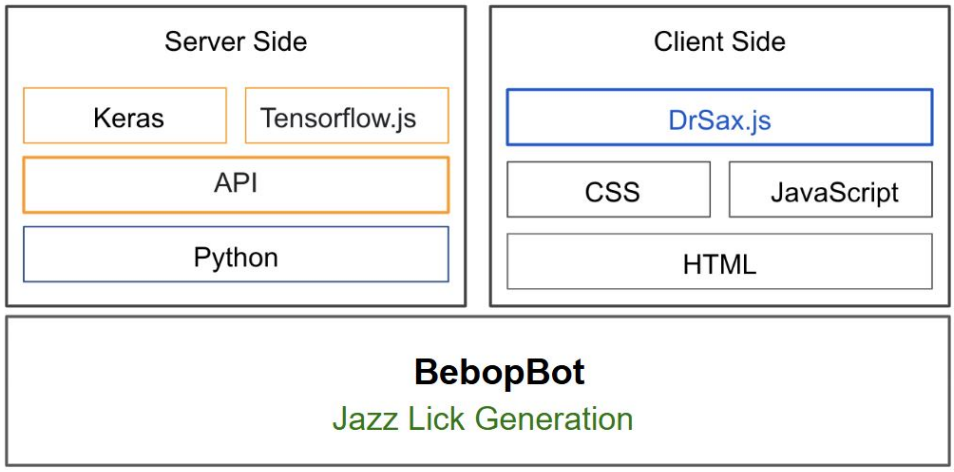


Fig. 3 Programming architecture of BebopBot.

System Operation Flow is as follows:

Generate Button → Input Data & API Call → Model Inference
→ Licks Generator → Response → Licks Storage
→ Notation Rendering → Playback Button
→ (Optional: Sound Synthesis & Timbre Control)
→ Audio Playback

When the user clicks the Generate button, three chord-tone-based input notes are sent to the RESTful API, which uses the trained model to generate an 8-note lick. As shown in Figure 2, the generated lick is displayed (in C7) in the musical notation in the middle of BebopBot. The generated lick is played using the Playback button with DrSax.js (Web Audio API library).

The sound consists of basic oscillator synthesis (default) and can be changed to FM and AM synthesis using the drop-down button. These synthesizer parameters can be adjusted using the two knobs above. Furthermore, the oscillator type can be changed using the OSC Type drop-down button. The Playback button is located below the musical notation and the slider next to it controls the volume. The generated licks (reused subsequently) are saved as string-type data just after generation.

4 Interaction Modeling

In this section, we explore interaction modeling scenarios between BebopBot and user.

- **Learning Scenario**
  Generate → Listen → Select → Practice → Improvisation
  (Practicing and utilizing the original generated jazz lick in jazz performance)
- **Motif Discovery**
  Generate → Listen → Discover Motif → Extract → Develop
  (Using part of the 8-note lick as a motif)
- **Lick Transformation**
  Generate → Select → Modify → Practice → Personal Lick
  (Extending the original jazz lick to create new licks)

5 Evaluation

We evaluate the performance of the BebopBot, focusing on accessibility, lick generation, interaction (for practicing and applying), usability, and musical analysis.

5.1 Accessibility

Users can access the BebopBot through web browser (on mobile and desktop), allowing users to generate a new jazz lick (with one click) and play the lick. It also enables real-time playback of generated licks (with the Web Audio API), allowing users to check the result without MIDI players or extra DAWs (Digital Audio Workstations).

These features support improved accessibility for practicing the lick.

5.2 Generation Efficiency

In jazz lick learning, users need to progress through several steps: searching for recordings, repeated listening, transcription, copying and practice. In contrast, BebopBot is designed to generate new licks with one click in a web browser.

Thus, users can easily obtain new musical ideas through the interaction modeling scenario (compare and select the licks through repeated generation). Therefore, BebopBot provides efficiency (in time and effort) for practicing jazz licks.

5.3 Musical Analysis

The generated licks produce one output based on the input pattern. Figure 4 shows the generated three licks with musical notation in BebopBot.

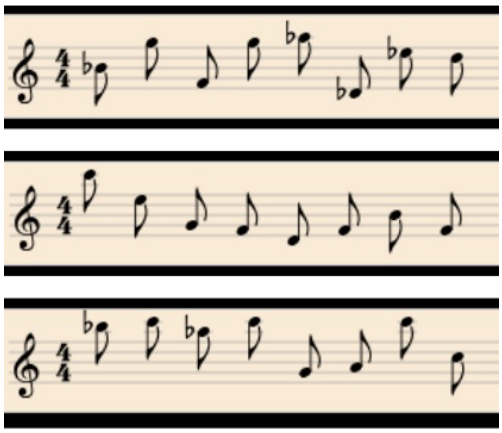


Fig. 4 The generated licks of BebopBot.

We analyze the first lick in Figure 4 as an example. The generated lick consists of Bb (7th), G (5th), F (11th), G (5th), Ab (b13), Db (b9), Eb (#9), D (9th). The melody contains both ascending and descending lines and includes the chord tones and tension notes. In particular, the b9, #9 and b13 tensions give an altered sound over C7.

Table 2 Musical Analysis of a Generated Lick in C7

Category	**Analysis**
Chord Tone	G (5th), Bb (7th)
Tension	D (9th), F (11th)
Altered Tension	Db (b9), Eb (#9), Ab (b13)

It shows the application of part of the jazz lick data for learning.

However, we should consider the following aspects for the generated jazz licks for musical and harmonic quality assessment. Also, the generated jazz licks are limited to short 8-note licks, which limits expressing longer licks. Besides, this system aims to explore the possibility of supporting new musical ideas for jazz learning and creation rather than jazz performance. We will extend this study to Transformer and RAG (like recent technologies); this study is meaningful as a preliminary study for extending the research.

6 Conclusion

This study proposed to develop a web-based jazz lick generator using web technologies and AI to reduce time and effort in the jazz lick learning process. This proposed system trains an LSTM model using a jazz dataset, and generated licks can be played in a web-based environment with DrSax.js. Also, users can access BebopBot on mobile and desktop, allowing them to repeatedly generate new licks. Users can learn the jazz licks through various interaction modeling scenarios based on these features in real jazz performance. As a result, BebopBot can generate 8-note licks without extra installation, and save the licks for learning and creation. The results show that combining the AI technology for music generation and web technology is one of the ways to support jazz improvisation learning.

But we need more studies on rhythm and expression. Also, there is a need for further assessment by the real jazz musicians. For future work, we aim to compare the LSTM, Transformer and RAG for music generation efficiency and aim to extend the support system for various rhythms and musical structures. Furthermore, we plan to develop the AI-based jazz performance system in a real jazz environment. We can expect that continuous research on jazz lick generation will develop into various forms for jazz performance and environment.

Declarations

Conflict of Interest

The author declares no conflict of interest.

Ethical Approval

Not applicable.

Author Contribution

The author conducted the study, designed the methodology and wrote the manuscript.

Funding

This research received no external funding.

Availability of Data and Materials

The datasets used in this study are available from the corresponding author upon reasonable request.

References

[1] Choi, H., Berger, J.: Waax: Web audio api extension. In: Proceedings of the International Conference on New Interfaces for Musical Expression, pp. 499–502 (2013)

[2] Hong, E., Kim, J.: Webxophone: Web audio wind instrument. In: Proceedings of the 1st International Conference on Algorithms, Computing and Systems, pp. 79–82 (2017)

[3] Werner, N., Balke, S., Stöter, F.-R., Müller, M., Edler, B.: trackswitch. js: A versatile web-based audio player for presenting scientific results (2017)

[4] Lee, S.W., Taylor, B., Essl, G., Filimowicz, M.: Interactive music on the web. Foundations in sound design for interactive media: a multidisciplinary approach, 200–228 (2019)

[5] Turchet, L., De Cet, M.: A web-based distributed system for integrating mobile music in choral performance. Personal and Ubiquitous Computing **27**(5), 1829–1842 (2023)

[6] Ma, S., Martin, C.: Touching wires: tactility and a quilted musical interface for human-ai musical co-creation. In: Proceedings of the International Conference on New Interfaces for Musical Expression, pp. 296–303 (2025)

[7] Smith, J.B., Freeman, J.: Adaptation and perceived creative autonomy in gesture-controlled interactive music. In: Proceedings of the 25th International Conference on New Interfaces for Musical Expression (2025)

[8] Aynsley, H., Bennett, P., Meckin, D., Hollowell, S., Mitchell, T.J.: Instant design: five strategies for the use of generative ai in nime ideation workshops. In: Proceedings of the International Conference on New Interfaces for Musical Expression, pp. 26–32 (2025)

[9] Wen, X., Li, W.: Time series prediction based on lstm-attention-lstm model. IEEE access **11**, 48322–48331 (2023)

[10] Al-Selwi, S.M., Hassan, M.F., Abdulkadir, S.J., Muneer, A., Sumiea, E.H., Alqushaibi, A., Ragab, M.G.: Rnn-lstm: From applications to modeling techniques and beyond—systematic review. Journal of King Saud University Computer and Information Sciences **36**(5), 102068 (2024)

[11] Alizadegan, H., Rashidi Malki, B., Radmehr, A., Karimi, H., Ilani, M.A.:

Comparative study of long short-term memory (lstm), bidirectional lstm, and traditional machine learning approaches for energy consumption prediction. Energy Exploration & Exploitation **43**(1), 281–301 (2025)

[12] Azimi, M., Zareei, M.H.: Live improvisation with fine-tuned generative ai: A musical metacreation approach. In: Proceedings of the International Conference on New Interfaces for Musical Expression, pp. 389–393 (2025)

[13] Hong, E., Kim, J.: Drsax. js: A javascript based unified web audio library and framework. In: Proceedings of the 1st International Conference on Algorithms, Computing and Systems, pp. 74–78 (2017)

[14] Jazzomat Research Project: The Weimar Jazz Database (WJazzD) (v1.4.1). https://jazzomat.hfm-weimar.de/. Hochschule für Musik FRANZ LISZT Weimar (2020)